

OS ASSIGNMENT6

2402MC07

Write ups for Q3 & Q4:

Question 3 — Deadlock Prevention via Resource Ordering

Overview:

Deadlock prevention works by invalidating at least one of the four necessary Coffman conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, or Circular Wait. This implementation prevents deadlock by eliminating **Circular Wait**.

Key Principles:

- **Out-of-Order Failure (`resourceorder_bad.c`):** Process A requests Lock 1 then Lock 2, while Process B requests Lock 2 then Lock 1. If Process A grabs Lock 1 while Process B grabs Lock 2, both processes wait indefinitely on each other, causing a deadlock (hang).
- **Hierarchical Solution (`resourceorder_fixed.c`):** A global total ordering is assigned to all resources (e.g., $\text{Lock 1} = 1$, $\text{Lock 2} = 2$). All processes are strictly required to request locks in ascending numerical order.
- **Why It Prevents Deadlock:** Because every process must acquire Lock 1 before requesting Lock 2, no process can acquire Lock 2 first. The process that wins Lock 1 will complete its work without getting blocked by another process holding Lock 2. A circular dependency becomes impossible.

Question 4 — Combined Synchronization & Deadlock Avoidance

Overview:

Extends resource management to a multi-instance setting (5 concurrent processes contending for 2 Printers, 1 Scanner, and 2 Disks) using counting semaphores built over xv6 IPC pipes.

Strategy and Implementation:

- **Synchronization:** Multi-instance resources are managed using pipe buffers. A process acquiring a resource reads a token from the pipe (blocking if empty), and puts the token back upon releasing the resource.
- **Deadlock Strategy (Global Resource Ordering):** Resource types are indexed strictly:
 - $\text{Printer} = 0$
 - $\text{Scanner} = 1$
 - $\text{Disk} = 2$
- **Execution Rule:** Even if a process needs a Disk and a Printer, it must request the Printer (0) first, and the Disk (2) second.
- **Outcome:** Enforcing this static global ordering across all 5 processes guarantees that the resource allocation graph remains acyclic throughout execution, allowing all processes to safely acquire resources, complete their iterations, and exit cleanly without deadlocks or resource leaks.